

C PROGRAMMING

Complete Foundation Guide

Introduction • Number Systems • Tokens • Data Types

Operators • Control Flow • Functions • Arrays • Strings

Pointers • Structures • File I/O • Dynamic Memory Allocation

From First Program to Dynamic Memory — A Complete Beginner's Journey

Table of Contents

Chapter 1	Introduction to C Programming
Chapter 2	Compiling & Running a C Program
Chapter 3	Number Systems
Chapter 4	Tokens in C
Chapter 5	Data Types & Variables
Chapter 6	Operators in C
Chapter 7	Control Flow Statements
Chapter 8	Functions
Chapter 9	Arrays
Chapter 10	Strings
Chapter 11	Pointers
Chapter 12	Structures & Unions
Chapter 13	File Handling
Chapter 14	Dynamic Memory Allocation

■ What is C?

C is a general-purpose, procedural programming language developed by **Dennis Ritchie** at Bell Labs between **1969 and 1973**. Originally designed to write the UNIX operating system, C has become one of the most widely used and influential programming languages in history.

C provides low-level access to memory through pointers, a clean and structured syntax, and efficient compiled performance — making it ideal for system software, embedded systems, game engines, and performance-critical applications.

■ Why Learn C?

- Foundation for C++, Java, Python, and nearly every modern language
- Direct hardware/memory control via pointers
- Extremely fast execution — compiled to native machine code
- Used in operating systems (Linux, Windows kernel), embedded systems, databases
- Helps you understand how computers actually work

■ History & Standards

Year	Event
1969	C developed from BCPL/B at Bell Labs
1972	First proper C implementation on PDP-11
1978	K&R; C — Kernighan & Ritchie book published
1989	ANSI C / C89 — First standardisation
1999	C99 — New features: // comments, flexible arrays
2011	C11 — Unicode support, multi-threading
2018	C17 — Bug fixes and clarifications
2023	C23 — Latest revision

■ Structure of a C Program

```
/* Basic Structure of a C Program */

#include <stdio.h>      // Header file (preprocessor directive)
#include <stdlib.h>

// Global declarations
int globalVar = 10;
```

```
// Function prototype
void greet(void);

// Main function - entry point
int main() {
    int localVar = 5;          // Local variable
    greet();                   // Function call
    printf("Value: %d\n", localVar + globalVar);
    return 0;                  // Return status to OS
}

// Function definition
void greet(void) {
    printf("Hello, World!\n");
}
```

■ *Note: Every C program must have exactly one main() function. Execution always starts here.*